Codexa Proof and Runtime Hardening

PR summary for `codex/enhance-features` against `main`.

Base	`834bd5e` (`v0.17.1`)
Source head before summary artifacts	`46104e9`
Source delta	68 files, 5,016 insertions, 474 deletions
Commits	3 Conventional Commits

Executive Outcome

The sweep concentrated on the highest-ROI trust boundaries: proof authority, verification credit, durable local state, real multi-language workflows, MCP coexistence, hot-path bounds, and release supply-chain integrity. The result is a more deterministic completion system that fails closed when evidence is ambiguous, survives concurrent and interrupted writers, and remains useful in ordinary Python, JavaScript, TypeScript, monorepo, nested-repository, and multi-MCP use.

The design follows large-scale functional-core principles: explicit state transitions, pure bounded matchers and classifiers, immutable evidence inputs, small policy modules, deterministic identities, and side effects isolated at validated persistence and process boundaries.

Problems Addressed

Codexa's completion authority could previously be weakened by stale or ambiguous proof, overly broad command credit, concurrent local receipt writers, damaged or redirected session state, and workspace identities that did not bind every raw byte. Verification recipes did not cover enough real-world runner and package layouts. MCP cleanup could also mistake an unrelated Graphify server for Codexa when its executable path happened to contain the word `codexa`, while release automation still depended on floating action and publisher downloads.

Highest-ROI Adjustments

1. Evidence and proof authority

- Reconstructs required test, workflow, and dependency checks from saved task

scope instead of allowing incomplete completion claims.

- Preserves safe repository-bound command handoffs while keeping previews,

reports, artifacts, failures, and waivers semantically distinct.

- Binds review authority to exact raw workspace content across dirty-to-commit

transitions, nested repositories, line-ending normalization, file-mode changes, and status-hidden edits.

- Credits only bounded, executable test scopes through command-classifier v6.

Python unittest discovery, Node test globs, Playwright, and Cypress are supported; filtered, unsafe, unmatched, ambiguous, and outside-repository invocations fail closed.

- Command-classifier v6 also keeps a bare `cypress run` uncredited because its

config-defined discovery cannot prove coverage of unrelated Jest/Vitest recommendations; one explicit indexed `--spec` remains target-only evidence, including conventional `*.cy.ts/js` Cypress filenames.

2. Real workflow coverage

- Generates runner- and package-aware candidate verification commands.
- Completes broad post-edit coverage in bounded passes.
- Serializes concurrent outcome-pointer and hook-journal publication with

collision-safe identities.

- Preserves exact proof context through commit, reopen, and re-index flows.

3. Durable session state

- Reconciles the write-ahead event log with the materialized session store.
- Rejects redirected, cross-session, and non-regular managed state.
- Recovers from interrupted log tails without trusting damaged evidence.
- Maps non-portable session identifiers to deterministic filesystem-safe names.

4. Bounded retrieval and portable artifacts

- Uses bounded compiled matcher caches with parity between search and retrieval

scoring.

- Applies shared raw-search limits and preserves the `git grep` fallback path.
- Gives generated module artifacts deterministic, collision-safe filenames even

when names collapse to the same slug or run on case-insensitive filesystems.

5. MCP and release integrity

- Preserves unrelated Graphify MCP configuration during Codexa initialization.
- Compiles the main development gate once instead of rebuilding for each test

phase.

- Pins GitHub Actions to immutable commits.
- Pins MCP Publisher by version and SHA-256 before extraction.

Verification

Deterministic release gate

`npm run security:check` passed on the source head:

- 102 test files passed.

- 1,186 tests passed; 1 intentionally skipped.
- 117 Claude command/hook integration smokes passed.
- npm audit reported 0 vulnerabilities.
- Clean public snapshot, package hygiene, plugin hygiene, startup-context

budget, and the 31-check packaged-install smoke passed.

- `git diff --check` passed.

Human workflow simulations

Four disposable repositories were changed, tested, committed, and reopened as a developer would use them:

- Python `src/` layout: 5 unittest cases passed with the repository-declared

`PYTHONPATH=src` recipe.

- JavaScript monorepo: 3 Node test cases passed across workspace packages.
- TypeScript package: 4 Vitest cases passed after a behavior change.
- JavaScript service with Graphify: 6 Node test cases passed after pricing and

order-validation changes.

Strict reopen checks correctly rejected two indexes whose HEAD changed after a commit, emitted the exact `codexa index` recovery action, and passed after the indexes were refreshed. The other two repositories reopened with fresh, identity-matched indexes. This exercises the intended fail-closed path rather than hiding staleness.

Graphify MCP interoperation

The same project configuration retained both Graphify and Codexa server blocks. Fresh stdio client sessions initialized both servers, listed their tools, and made real calls:

- Graphify `graph_stats`: 26 nodes and 32 edges; no tool error.
- Codexa `search("shipping threshold")`: returned

`raw_search_sufficient` and a content-addressed detailed result; no tool error.

The current Graphify Python environment required an `mcp<2` constraint because of Graphify's upstream dependency compatibility. That workaround is independent of Codexa's Node runtime and is not included in the package.

Risk-Budgeted Review

Finding weights are critical 8, high 5, medium 3, and low 1. Merge requires zero critical/high findings and no more than 4 total residual points. An independent final-head review covers correctness, security, data integrity, performance bounds, compatibility, and release operations.

The first independent pass found one high-severity over-credit path: a bare `cypress run` could cover unrelated JavaScript recommendations. Commit `46104e9` closes it fail-closed, preserves explicit `--spec` credit, adds conventional `*.cy.ts/js` recognition, bumps classifier provenance, and adds direct/script/candidate-command regression coverage. Re-review found no remaining critical, high, medium, or low findings: residual score 0, merge recommended.

Operational Watch Items (Not Scored Findings)

- Stricter classification can turn previously credited ambiguous commands into missing evidence; callers must run an explicit target or provide a state-bound verification manifest.
- Exact identity and review scans are bounded, so unusually large or actively mutating worktrees fail closed instead of claiming readiness.
- Session reconciliation can surface recovery warnings for damaged legacy local state rather than silently trusting it.
- Commit-pinned actions and publisher tools require deliberate maintenance updates.
- Graphify's Python MCP compatibility issue remains upstream.

Rollout and Deployment

1. Require the GitHub check, package smoke, benchmark, and native Ubuntu/macOS/Windows worktree-bootstrap lanes on the exact PR head.
2. Merge through the protected branch only after the final risk-budget review is within budget.
3. Use the repository's canonical secret-backed `release-please.yml` lane with `RELEASE_PLEASE_TOKEN` to create or update the release PR; verify its version files, changelog, and exact-head checks before merging it.
4. Let the resulting GitHub Release trigger `npm-publish.yml` with the repository's `NPM_TOKEN`; monitor npm provenance and MCP Registry publish.
5. Live-verify the GitHub tag and release, npm registry version, clean package installation, CLI version/startup, and MCP server initialization/tool call.

Rollback

Before publication, revert the feature commits and regenerate local Codexa cache/state as needed. After npm publication, do not overwrite or silently yank the immutable version: revert on `main`, publish a corrective patch, and deprecate the affected version only if its behavior is materially unsafe.

Summary Artifacts

- Markdown: `docs/pr-summaries/codex-general-codexa-20260803-enhance-features/summary.md`
- PDF: `docs/pr-summaries/codex-general-codexa-20260803-enhance-features/summary.pdf`